COMPLETE TECHNICAL DOCUMENTATION

MIRQAB from A to Z

Architecture, components, data model, pipelines, security, testing, deployment and operations — describing the repository as it exists on 27 September 2026.

TEAM

Bdoor Fahed Alosaimi
Nouf Saleh Altalhi
Alaa Rafa Al Saad

PROJECT

MIRQAB — AI-Powered Security Operations Platform for SMBs

27 September 2026

Contents

1. Overview
2. Languages and technologies
3. Architecture
4. Repository structure
5. Frontend — SOC Dashboard
6. Frontend — Employee Desktop
7. Backend
8. Database, Prisma and PostgreSQL
9. Redis
10. Wazuh (Agent, Manager, Indexer, Dashboard)
11. Nginx
12. WebSockets
13. Incident pipeline
14. Detection
15. Correlation
16. MITRE ATT&CK
17. IOC / IOA
18. Evidence
19. Incident actions
20. Device isolation
21. Notifications
22. Digital Twin
23. Reports (case and weekly)
24. Simulation
25. AI architecture and self-hosted AI
26. Security, RBAC, multi-tenancy
27. Testing and QA
28. Deployment
29. Backup and disaster recovery
30. Monitoring
31. Implementation status
32. Known limitations
33. Roadmap
34. Glossary

How to read the status labels

VERIFIED	Tested and confirmed: by the automated test suite (38 suites, 421 tests, all passing on 27 Sep 2026) and/or by a live test recorded in the team's engineering reports.
VERIFIED IN LAB	Confirmed end-to-end in a controlled, containerized lab. Not tested on physical employee devices.
LIMITED	Implemented and working, with a clearly stated limitation.
NOT VERIFIED	Implemented in code but not tested live.
NOT IMPLEMENTED	Not available in the application.
NOT DEPLOYED	No production deployment exists.
BLOCKED	Cannot be completed because of an external dependency.
PLANNED	A future roadmap item. Not built.
DEMO / SIMULATION	Uses synthetic or public demo data. Not a real environment.

1. Overview

MIRQAB (مرقاب, "watchtower") is a Security Operations Center (SOC) platform for small and medium-sized businesses. It ingests security events, turns them into correlated and explainable incidents, guides analysts through investigation, enforces a 5-minute response window for HIGH/CRITICAL incidents, and contains endpoints through Wazuh Active Response, trusting only agent-confirmed results.

Team	Bdoor Fahed Alosaimi · Nouf Saleh Altalhi · Alaa Rafa Al Saad
Applications	SOC Dashboard (web), Employee Desktop (Electron), Backend API
State	Working prototype running locally in Docker. **NOT DEPLOYED** to production.
Tests	38 backend suites / 421 tests passing; clean TypeScript checks for all three apps (27 Sep 2026)

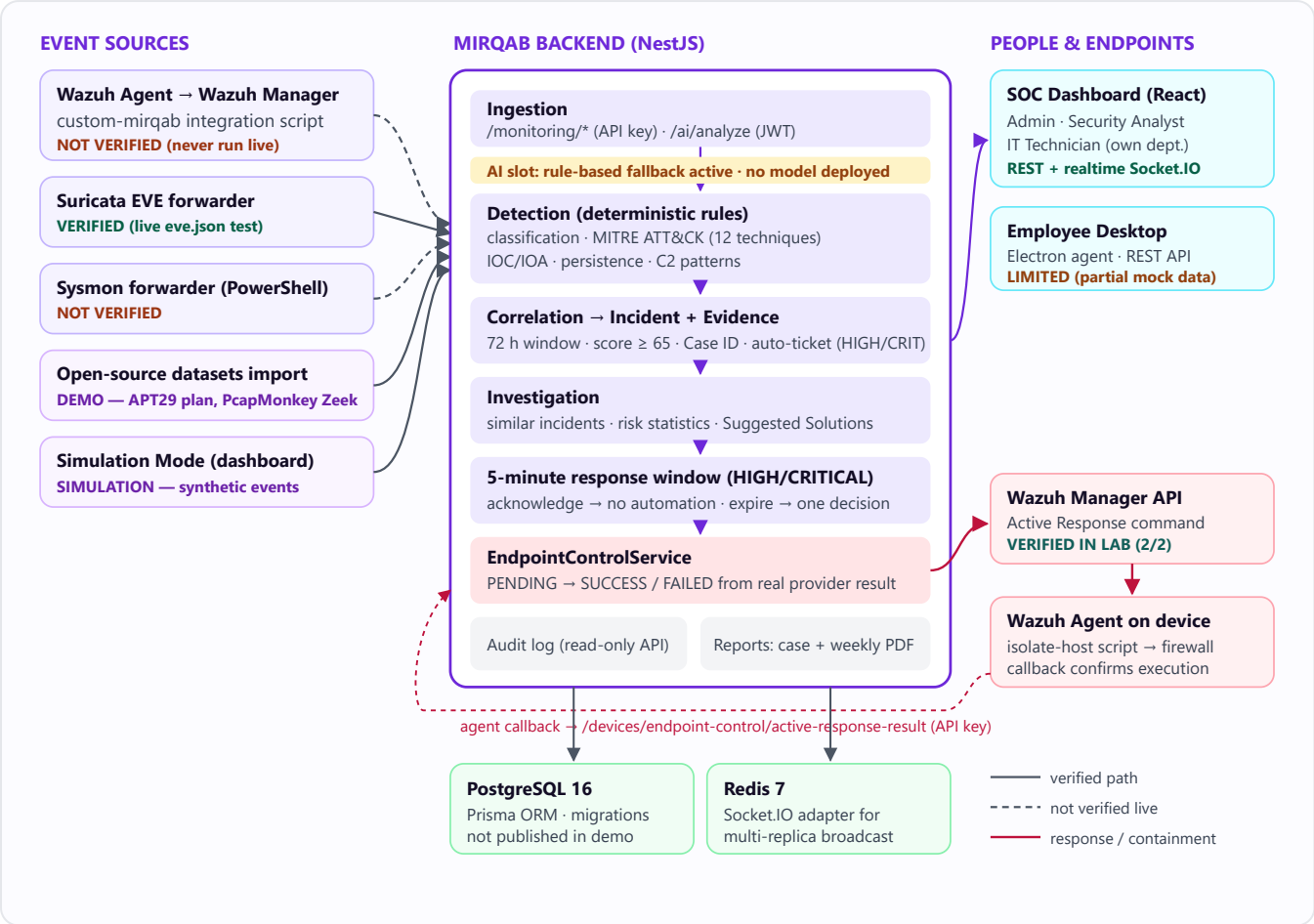
2. Languages and technologies actually present

Language / format	Where	Files*
TypeScript (`.ts` , `.tsx`)	Backend, SOC Dashboard, Employee Desktop	274 + 76
JavaScript (`.cjs` , `.js`)	Playwright QA scripts, Suricata EVE forwarder	29 + 3
SQL	Prisma migrations	14
PowerShell	Sysmon forwarder, Windows isolation script, test scripts	7
Shell (bash/sh)	Linux isolation scripts, container entrypoint, certificate generator	4
Python	Wazuh Manager integration script (`mirqab-integration.py`)	1
HTML / CSS, YAML, XML, nginx config	App shells and styles, Docker Compose, Wazuh config snippets, gateway config	—

* Counts of files in the clean source package (excluding dependencies and build output).

Layer	Technologies (major versions from package manifests)
Backend	Node.js 20 (container), NestJS 10, Prisma 5, PostgreSQL 16, Redis 7 (ioredis), Socket.IO 4 + Redis adapter, Passport JWT, bcryptjs, class-validator, Joi, Helmet, @nestjs/throttler, @nestjs/schedule, @nestjs/event-emitter, PDFKit, Swagger (non-production only), Anthropic SDK (dev-only, disabled by default)
SOC Dashboard	React 18, Vite 5, TypeScript 5, Tailwind CSS 3, TanStack React Query 5, Zustand, React Router 6, Recharts, React Flow, Framer Motion, Socket.IO client, Axios, lucide icons
Employee Desktop	Electron 29, React 18, Vite 5, Tailwind CSS 3, React Query, Zustand, electron-builder
Infrastructure	Docker, Docker Compose, nginx 1.25, PostgreSQL 16 (alpine), Redis 7 (alpine)
Security telemetry	Wazuh (Manager API, Active Response, agent scripts), Suricata EVE JSON, Sysmon (forwarder)
Testing	Jest 29 + ts-jest (backend), Playwright (UI QA scripts), TypeScript compiler checks

3. Architecture



The backend is the single source of truth. Both frontends talk only to its REST API, and the SOC Dashboard also receives server-to-client events over Socket.IO. Event sources reach the backend through authenticated ingestion endpoints. Containment leaves the backend only through `EndpointControlService`. See the Technical Architecture document for component diagrams and the planned-versus-actual comparison.

4. Repository structure

```
mirqab-project/
├── apps/
│   ├── soc-dashboard/        React + Vite web app (+ e2e/ Playwright QA scripts)
│   └── employee-desktop/     Electron + React endpoint agent
├── backend/                  NestJS API
│   └── src/
│       ├── modules/          ai, alerts, audit-logs, auth, demo, departments, devices,
│       │                     incident-memory, incidents, monitoring, notifications,
│       │                     policies, reports, security-testing, simulation,
│       │                     threat-intel, tickets, users, websocket
│       ├── common/           decorators, filters, guards, interceptors, pipes, rbac
│       ├── config/           configuration + environment validation (Joi)
│       ├── database/         prisma/schema.prisma, prisma/migrations (14), seed/
│       └── redis/
├── infra/
│   ├── docker/               docker-compose.yml, nginx (TLS), redis, postgres notes
│   ├── demo/                 evaluator demo: docker-compose.demo.yml, nginx-demo.conf
│   └── monitoring/           wazuh (integration + active-response), suricata, sysmon
├── datasets/                 small public samples (see Demo Data document)
├── docs/                     engineering phase notes and integration notes
├── packages/                 placeholders for shared types / UI kit (empty)
├── ai-engine/                earlier standalone design — unused (AI lives in backend)
└── scripts/                  PowerShell test scripts
```

5. Frontend — SOC Dashboard

React 18 single-page application, served as static files by an nginx container and reached through the gateway. Arabic (RTL) is the default language, and English is available through a switch. It has dark and light themes.

Route	Page	Purpose
`/`	Overview	KPIs, live network map, active incidents, analytics charts, Simulation Mode start
`/alerts`	Incidents	Filterable incident list; opens the two-window incident view
`/threat-map`	Live Network Map	Topology of departments and devices with active attacks
`/citadel`	The Citadel — Live View	Wallboard view, including the APT29 Attack Path tab
`/departments`, `/devices`	Departments, Devices	Inventory with risk and security scores
`/notifications`	Notifications	In-app notifications with filters
`/tickets`	Tickets	Auto-created and manual tickets
`/incident-memory`	Incident Memory	Past incidents and similarity

Route	Page	Purpose
`/reports`	Reports	Weekly SOC reports: generate, list, download
`/analytics`	Analytics	Trends and distributions
`/policies`	Policies & Rules	Including the "5-Minute Response Window" kill switch
`/users`, `/audit-logs`, `/settings`	User Management, Audit Logs, Settings	Administration
`/security-testing`	Security Testing Mode	Non-destructive self-checks
`/login`, `/register`, `/forgot-password`, `/reset-password`	Authentication	Account access

Data access goes through a service/adapter/hook layer built on React Query. Realtime events trigger cache invalidations (never direct cache writes), with 30-second polling as a safety net.

6. Frontend — Employee Desktop

An Electron app for employee devices: login, device health (CPU, RAM, disk, Defender, firewall, processes and apps read through the operating system), security score, device info, AI scan, notifications, incident history and recommended fixes. Its refresh token is stored encrypted. **LIMITED**: the AI scan classification and sample incident history use clearly-marked mock data, and the app is not packaged for distribution.

7. Backend

A NestJS application with the global prefix `/api/v1`. Global protections: Helmet, a validation pipe (whitelist and forbid unknown fields), a null-byte rejection pipe, a uniform exception filter, and a throttler (100/min default). Swagger is mounted only outside production. Environment variables are validated at startup, for example JWT secrets of at least 32 characters.

Module	Responsibility
`auth`, `users`	Login, signup, refresh rotation, password reset, user management
`ai`	Analysis pipeline, MITRE mapping, enrichment, correlation, Case IDs, similarity engine, risk statistics, agents, response window (`response-sla`), recommendations, provider registry
`monitoring`	`/monitoring/wazuh|suricata|sysmon|pcap` ingestion (API key), dataset sample importers, Wazuh logtest enrichment
`devices`	Devices and `endpoint-control` (Wazuh Active Response provider, generic EDR provider, callback endpoint)
`incidents`, `alerts`, `incident-memory`, `threat-intel`	Incident-related domain services
`tickets`, `policies`, `departments`	Domain modules
`notifications`, `websocket`	Notifications and the `/realtime` Socket.IO gateway
`audit-logs`	Read-only audit API
`reports`	Case report PDF, weekly report PDF and its scheduler
`security-testing`	Security Testing Mode
`simulation`, `demo`	Simulation support; `DemoDataService` for the evaluator demo (`MIRQAB_DEMO_DATA=true`)

8. Database, Prisma and PostgreSQL

PostgreSQL 16, accessed only through Prisma 5. The schema lives in `backend/src/database/prisma/schema.prisma`, and there are **14 migrations**. On every backend container start, the entrypoint runs `prisma migrate deploy` (applying only missing migrations), then an idempotent seed that creates the organization, admin, departments, devices and policies only if they are missing, and then starts the server.

Main tables: `organizations`, `users`, `refresh_tokens`, `password_reset_tokens`, `departments`, `devices`, `incidents`, `incident_evidence`, `incident_actions`, `indicators`, `persistence_findings`, `ai_agent_decisions`, `incident_memory`, `case_counters`, `tickets`, `policies`, `notifications`, `audit_logs`, `weekly_reports`.

Key enums: roles (ADMIN, SECURITY_ANALYST, IT_TECHNICIAN, EMPLOYEE); severity (LOW, MEDIUM, HIGH, CRITICAL); incident status (OPEN, INVESTIGATING, CONTAINED, RESOLVED, CLOSED); event source (LIVE_AGENT, OPEN_SOURCE_DATASET, SIMULATION, MANUAL_TEST); action types (ISOLATE_DEVICE, DISABLE_ACCOUNT, BLOCK_IP, BLOCK_HASH, RESET_PASSWORD, UPDATE_FIREWALL, KILL_PROCESS, REMOVE_MALWARE, COLLECT_FORENSICS, MONITOR_DEPARTMENT); action outcome (PENDING, SUCCESS, FAILED).

A fresh database produced by the migrations matches `schema.prisma` exactly. This was verified on 27 Sep 2026 with `prisma migrate diff`, which reported an empty diff.

9. Redis

Redis 7 backs the Socket.IO Redis adapter, so realtime events reach clients connected to **any** backend replica when the backend is scaled (`docker compose up --scale backend=N`). Without `REDIS_URL` the gateway runs single-instance. Redis holds no primary data.

10. Wazuh

Component	Role in MIRQAB	Status
Wazuh Agent	Runs on the endpoint. It executes MIRQAB's registered Active Response script (`isolate-host.sh` / `.ps1`), which changes the firewall and calls MIRQAB back.	VERIFIED IN LAB (Linux)
Wazuh Manager	(a) Could forward alerts to MIRQAB through the `custom-mirqab` integration script. (b) Its REST API is called by MIRQAB to dispatch Active Response and, optionally, for *logtest* enrichment.	(a) NOT VERIFIED · (b) VERIFIED IN LAB
Wazuh Indexer	Part of the upstream Wazuh single-node stack used in the lab. **MIRQAB does not read from or write to the Indexer.**	NOT INTEGRATED
Wazuh Dashboard	Wazuh's own web UI in the upstream stack. **Not integrated**: MIRQAB has its own SOC Dashboard.	NOT INTEGRATED

The lab used the upstream `wazuh/wazuh-docker` single-node deployment with Wazuh 4.14.7, which the Docker Hub tag served as a **release candidate** build (BLOCKED for GA). That upstream checkout is not part of the submitted source.

11. Nginx

Development stack (`infra/docker/nginx`): HTTPS on port 443 with a self-signed certificate, and HTTP on port 80 redirecting to HTTPS. `/api/v1/` and `/socket.io/` are proxied to the backend (re-resolved through Docker DNS, so scaled replicas are all used), and everything else to the SOC Dashboard container. It adds five security headers. **Evaluator demo** (`infra/demo/nginx-demo.conf`): the same routing over plain HTTP, published only on `127.0.0.1:8080` .

12. WebSockets

Socket.IO namespace `/realtime`, JWT-authenticated handshake, dashboard roles only, department- and organization-scoped rooms, and no client-to-server messages. Server events: `incident.created`, `incident.updated`, `incident.evidence-added`, `incident.persistence-detected`, `ticket.created`, `ticket.updated`, `policy.updated`, `notification.created`, `audit-log.created`. Domain services emit events through an internal event emitter, and the gateway is the only listener.

13. Incident pipeline

1. **Ingest.** `POST /monitoring/{wazuh|suricata|sysmon|pcap}` (API key) normalizes each vendor format into a log excerpt. `POST /ai/analyze` (JWT, 20/min) accepts excerpts from Simulation Mode (source `SIMULATION`) or manual tests (`MANUAL_TEST`). Dataset importers use source `OPEN_SOURCE_DATASET`.
2. **Classify.** The rule-based classifier (or a configured AI provider, validated) assigns severity and a MITRE technique.
3. **Enrich.** IOC/IOA, persistence and C2 extraction. Optional Wazuh logtest enrichment.
4. **Correlate.** Join an open case (72 h, score ≥ 65) or create a new incident with a Case ID (`INC-YYYYMMDD-NNNN`).
5. **Persist.** Incident, evidence (with source label and raw excerpt), indicators, findings and agent decisions.
6. **React.** HIGH/CRITICAL: auto-ticket, "response required" notification and a 5-minute response window. Emit realtime events.
7. **Investigate and respond.** Similar incidents, risk statistics, Suggested Solutions, response actions, status changes, resolution record.
8. **Report.** Case report PDF on demand, and the weekly report on schedule.

14. Detection

Deterministic keyword and pattern rules classify events and set severity. Separate detectors cover persistence (scheduled tasks, services, startup folders, run keys, WMI, login scripts, cron, PowerShell profiles, account changes) and command-and-control beacons. VERIFIED · LIMITED coverage.

15. Correlation

Implemented in `ai/correlation`. It reuses the incident comparison engine (dimensions: MITRE technique and tactic, indicators, malware, device, department, timeline). Constants: window 72 hours, threshold 65. Only OPEN, INVESTIGATING and CONTAINED cases are eligible, so resolved and closed cases start fresh. VERIFIED

16. MITRE ATT&CK

A curated reference of 12 techniques in `ai/mitre-attack/mitre-reference.data.ts`: T1566, T1110, T1204, T1046, T1068, T1486, T1071, T1053, T1078, T1059, T1105 and T1567. Mapping is keyword-based, and unmatched events are `UNCLASSIFIED`. Any AI-proposed technique ID would be validated against this list. VERIFIED LIMITED

17. IOC / IOA

`ai/detection/indicator.util.ts`: IOCs (IP, HASH, DOMAIN, URL, EMAIL) with per-type confidence, and IOAs (BEHAVIOR subtypes: credential-dumping, lateral-movement, process-injection, obfuscated-execution, defense-evasion). Nothing is fabricated: no match means no row. VERIFIED

18. Evidence

Each contributing event becomes an `incident_evidence` row with its raw excerpt, source (`ingestSource`, plus `datasetOrigin` for datasets) and correlation score. The panel labels evidence "Live Agent", "Open-Source Dataset", "Simulation (Synthetic)" or "Manual / Test".

19. Incident actions

`incident_actions` records response actions (10 types) with outcome PENDING, SUCCESS or FAILED, a server-derived actor, notes, and approval state for high-impact recommended actions (AI may recommend, never execute). Analysts can log actions, update outcomes, approve or reject recommendations, and "Apply Suggested Solutions".

20. Device isolation

See the Security documentation, section 13. The flow is authorize, then confirm, then save as PENDING, then call the provider (Wazuh Active Response or a generic EDR API), then wait for the agent callback before recording SUCCESS or FAILED. The `ENDPOINT_CONTROL_API_URL` contract: `POST {url}/isolate` with a bearer key and a JSON body `{hostname, ipAddress}`, expecting `{success, detail?}`. **VERIFIED IN LAB** (Wazuh). The generic EDR provider is **NOT VERIFIED** against a real product.

21. Notifications

Notification kinds: `RESPONSE_REQUIRED` and `CONTAINMENT_RESULT` (SUCCEEDED, FAILED, UNAVAILABLE, POLICY_DISABLED), plus older generic notifications. Recipients are exactly the SOC users who can see the incident. The dashboard renders localized text from structured fields. VERIFIED · LIMITED: no per-employee notifications.

22. Digital Twin

Live Network Map and the Overview topology (React Flow) render internet → firewall → core switch → application and database servers → departments → devices. Nodes are coloured by active incidents, and a device opens its incident. *The Citadel* provides a wallboard view and an Attack Path tab for the public APT29 emulation plan (static client-side data, DEMO). It is built from MIRQAB records, not from network scanning. VERIFIED (security Phase 14)

23. Reports

Case report (`GET /reports/case/:id`): a PDFKit document with 11 sections, access-controlled like the incident. **Weekly report**: scheduled every Monday 05:00 UTC for the just-completed Sunday–Saturday week and each organization; endpoints `POST /reports/weekly/generate` , `GET /reports/weekly` , `/weekly/summary` , `/weekly/available-weeks` , `/weekly/:id/download` . VERIFIED

24. Simulation

Simulation Mode (dashboard) generates synthetic events for 5 scenarios — phishing (T1566), brute force (T1110), ransomware (T1486), port scan (T1046) and privilege escalation (T1068) — against existing devices. It posts them through the **real** `/ai/analyze` pipeline with source `SIMULATION` . Excerpts use reserved documentation IP ranges (RFC 5737). Simulated incidents are labelled "Simulation (Synthetic)" and are excluded from automated containment. SIMULATION

25. AI architecture and self-hosted AI

- **Provider registry:** a single active slot, resolved at startup and never chained. Candidates are the self-hosted provider (`SELF_HOSTED_AI_URL` , the intended production path) and Claude (development only: requires `ANTHROPIC_API_KEY` **and** `ALLOW_EXTERNAL_AI_DEV_ONLY=true`). With neither configured, which is the current and demo state, every AI-eligible task uses its rule-based fallback.
- **Self-hosted protocol:** the OpenAI-compatible `/v1/chat/completions` format (llama.cpp, Ollama, vLLM, LocalAI). Every reply is re-validated as untrusted text.
- **Guardrails:** prompt-injection delimiting, output schema, severity and MITRE validation, timeouts, circuit breaker, concurrency cap. Status visible at `GET /ai/provider-status` .
- **Agents:** INCIDENT_AGENT, IOC_IOA_AGENT, MITRE_RISK_AGENT and RECOMMENDATION_AGENT, each a declared least-privilege wrapper. None can create actions or reach endpoint control.
- **Risk prediction:** statistics from the department's own history (lateral-movement probability, spread time, affected devices, business impact, with sample size). Returns "insufficient data" when there is no history. It is not machine learning.
- **Incident memory:** a native vector similarity engine (FAISS IndexFlatIP-equivalent), department-scoped.

Status: architecture VERIFIED · self-hosted model NOT VERIFIED NOT DEPLOYED · external AI NOT USED AT RUNTIME .

26. Security, RBAC and multi-tenancy

Summarized here and detailed in the Security documentation: JWT HS256 pinning, refresh rotation with reuse detection, bcrypt, rate limits, four roles, department scoping, organization scoping (fail-closed), object-level guards, strict validation, read-only audit log, authenticated realtime, nginx TLS and headers, non-root backend container.

27. Testing and QA

Kind	What	Result
Unit / integration (Jest)	38 backend suites	**421 / 421 passing** (27 Sep 2026)
Type checks	Backend, SOC Dashboard, Employee Desktop	**0 errors** (27 Sep 2026)
UI QA (Playwright)	27 scripts in `apps/soc-dashboard/e2e`, run manually during development; screenshots in this package	Manual; no CI
Security phases	18 phases, live tests in several	See Security documentation
Security Testing Mode	Non-destructive in-app self-checks	Available in the app
Migration tests	Reconciliation migration tested on legacy, current and fresh databases (throwaway containers)	**Passed** (27 Sep 2026)
Evaluator demo, end-to-end	Full `infra/demo` stack from the packaged zip using the documented one command. 25 API checks, 4 realtime checks, 16 browser checks (real Chrome) and 3 response-window checks, followed by teardown.	**48 / 48 passed** on the dev machine (27 Sep 2026) · clean machine **NOT VERIFIED**
Load / performance	—	**NOT VERIFIED**

28. Deployment

- **Evaluator demo** (`infra/demo`): self-contained Compose project `mirqab-demo` with its own volumes, localhost-only on port 8080, safe configuration (no endpoint control, no external AI), and public datasets imported automatically. See the Demo Run Instructions.
- **Development stack** (`infra/docker`): PostgreSQL, Redis, backend, dashboard and nginx with TLS on ports 80/443. It supports `--scale backend=N` , which was verified live with multiple replicas during development.
- **Production:** **NOT DEPLOYED** . It would require a real domain, a CA certificate, managed secrets, backups, monitoring, and hardening of the published ports (see Security documentation, section 17).

29. Backup and disaster recovery

There is no automated backup or disaster-recovery tooling in the project **NOT IMPLEMENTED** . Data lives in the named Docker volume for PostgreSQL. A manual logical backup can be taken with `pg_dump` from the database container and restored with `pg_restore` . The team used exactly this on 27 Sep 2026, before applying a database migration, and validated restores in throwaway containers. Redis holds no primary data. Automated backups and a documented recovery objective are **PLANNED** .

30. Monitoring

Operational visibility is currently limited to container health checks (PostgreSQL, Redis), structured application logs (`docker logs`), the AI provider status endpoint, and in-app audit logs and notifications. There is no metrics or alerting stack (for example Prometheus or Grafana) for MIRQAB itself **NOT IMPLEMENTED** .

31. Implementation status

Capability	Status	Evidence and notes
Overall deployment	NOT DEPLOYED	MIRQAB runs locally in Docker on the team's development machine. There is no production deployment, public domain or CA-issued TLS certificate (a self-signed development certificate is used).
Backend API (NestJS)	VERIFIED	38 test suites / 421 tests passing and a clean TypeScript check on 27 Sep 2026. Running locally in Docker.
SOC Dashboard (React web app)	VERIFIED	Clean TypeScript check. Rebuilt from the current source and served at `https://localhost/` on 27 Sep 2026.
Employee Desktop agent (Electron)	LIMITED	Clean TypeScript check. Reads real device telemetry. Some screens (AI scan, sample incident history) show clearly-marked mock data. Not packaged or distributed.
Authentication and sessions	VERIFIED	JWT pinned to HS256, refresh-token rotation with reuse detection, bcrypt password hashing, rate-limited login. Unit tests plus live tests (security Phase 11).
Role-based access control and department scoping	VERIFIED	4 roles. 18 of 18 live cross-department / IDOR tests passed (security Phase 1), plus unit tests.
Organization (tenant) isolation	VERIFIED LIMITED	Enforced in the access guards and in every scoped query, and covered by unit tests. No live test with several organizations is recorded.
Wazuh alert ingestion endpoint	VERIFIED	`POST /monitoring/wazuh` tested live with Wazuh-format alert payloads. Each one created a real incident.
Wazuh Manager integration script	NOT VERIFIED	The `custom-mirqab` script that forwards alerts from a Wazuh Manager was reviewed but never run against a real manager.
Suricata EVE forwarder	VERIFIED	Run live against a growing `eve.json` file. It created a real incident.
Sysmon forwarder	NOT VERIFIED	Sysmon was not installed on the development machine.
Device isolation via Wazuh Active Response (Linux script)	VERIFIED IN LAB	2 of 2 end-to-end isolations through the MIRQAB application against a containerized safe-test Wazuh agent. A real firewall rule was applied and confirmed by an agent callback, measured at 11–15 s (security Phase 2A). Not tested on physical devices.
Windows isolation script	NOT VERIFIED	`isolate-host.ps1` exists but has not been run against a real Windows agent.
Releasing an isolated device (un-isolate)	NOT IMPLEMENTED	The application has no "release isolation" action. A companion script exists but was never deployed or tested.

Capability	Status	Evidence and notes
Wazuh general-availability version 4.14.7	BLOCKED	When checked (security Phase 4), the upstream 4.14.7 image tag still served release-candidate builds. The lab used that build.
5-minute response window	VERIFIED	Countdown, acknowledgement and once-only decision are covered by unit tests. The UI is shown in QA screenshots from 22–26 Sep 2026.
Automated containment when the window expires	LIMITED	The decision logic is unit-tested. In the development environment, automated attempts were correctly recorded as **FAILED** ("fetch failed") because no reachable endpoint was behind the device. A successful automated isolation against a live agent is not recorded.
Simulation safety (no real isolation for simulated incidents)	VERIFIED	Incidents whose source is `SIMULATION` are excluded from automated containment (unit-tested). The evaluator demo also hard-codes every endpoint-control setting to empty.
MITRE ATT&CK mapping	VERIFIED LIMITED	Deterministic, rule-based mapping to 12 curated techniques, unit-tested. Events that match no rule stay `UNCLASSIFIED` (11 of 27 demo incidents).
IOC / IOA extraction	VERIFIED	Deterministic extraction of IPs, hashes, domains, URLs and emails, plus 5 behaviour patterns. Nothing is invented: no match means no indicator.
Incident correlation	VERIFIED	Events join an open case within a 72-hour window when their similarity score is 65 or higher. Unit-tested.
Incident memory and similar-incident search	VERIFIED	Native vector similarity (equivalent to a FAISS flat inner-product index), department-scoped. Unit-tested.
Risk prediction	LIMITED	Statistics calculated from the department's own past incidents. It is not a machine-learning forecast, and it returns "insufficient data" when there is no history.
Suggested Solutions	VERIFIED	Fixes from a per-technique catalog plus actions that worked on similar past incidents. High-impact actions need analyst approval. Unit-tested.
External AI (Anthropic Claude)	NOT USED AT RUNTIME	Disabled by default. It is a development-only comparison option that needs both an API key and an explicit dev-only flag. It is off in the evaluator demo.
Self-hosted / private AI provider	NOT VERIFIED NOT DEPLOYED	The provider interface is implemented and tested, but no local model has been deployed (no suitable hardware). All AI-labelled analysis currently uses deterministic rules.
Notifications for SOC staff	VERIFIED	"Response required" and "containment result" notifications are scoped like the incidents they describe. Unit-tested.

Capability	Status	Evidence and notes
Notifications to individual employees	LIMITED	The backend has no per-employee ownership model. Notifications go to SOC roles (Admin, Security Analyst, IT Technician).
Live network map / Digital Twin	VERIFIED	Topology is built from real device, department and incident records, and its security was reviewed (security Phase 14). The APT29 attack-path view uses static public dataset data DEMO.
Case report (PDF)	VERIFIED	Generated per incident. Unit-tested, including access control.
Weekly SOC report (PDF)	VERIFIED	Scheduled every Monday at 05:00 UTC, plus manual generation. Unit-tested.
Arabic / English interface	LIMITED	Full language switch with right-to-left layout, but some incident-panel labels still appear in English in Arabic mode.
Dark / light theme	VERIFIED	Both themes are shown in QA screenshots.
Evaluator demo stack (`infra/demo`)	VERIFIED LIMITED	On 27 Sep 2026 the full demo stack was run end-to-end from the packaged source zip, using the documented one command, on the development machine. All 5 services were healthy, 22 + 6 dataset events were imported, and all 48 checks passed (API, RBAC, realtime, reports, browser walkthrough, response window). **It has not been run on a separate clean machine.**
Load / performance testing	NOT VERIFIED	No load test was performed. Query-count improvements were verified with unit tests only (security Phase 16).

32. Known limitations

1. **Not deployed to production.** MIRQAB runs locally in Docker on the development machine. It has no public domain, and TLS uses a self-signed development certificate.
2. **No runtime AI model.** Classification, MITRE mapping, risk estimates and recommendations are deterministic and rule-based. The self-hosted AI provider is implemented but **not verified**: no model has been deployed. External AI (Claude) is disabled by default and is never used in the demo.
3. **Device isolation was verified only in a lab.** 2 of 2 successful end-to-end isolations used a containerized Linux safe-test agent. There are no tests on physical employee devices, the Windows script is not verified, and releasing an isolated device is not implemented in the application.
4. **Automated containment has not been shown to succeed live.** When the 5-minute window expires with no response, MIRQAB records an isolation request. In the development environment those attempts were recorded as **FAILED**, because no reachable endpoint existed behind the device.
5. **Wazuh version is BLOCKED.** The upstream 4.14.7 image was still a release-candidate build when checked. The lab ran that build.
6. **Wazuh Manager → MIRQAB forwarding script not verified.** The ingestion endpoint was verified with Wazuh-format payloads, but the Manager-side script was never run. MIRQAB does **not** read from the Wazuh Indexer.
7. **Limited detection coverage.** 12 curated MITRE ATT&CK techniques. Events that match no rule stay `UNCLASSIFIED`. For example, the dataset notes report all 6 PcapMonkey network events landing as UNCLASSIFIED / LOW.
8. **Notifications target SOC staff, not individual employees.** The Employee Desktop app partly uses clearly-marked mock data.
9. **Partial Arabic translation** in some incident-panel labels.
10. **No load testing**, and no live multi-organization test.
11. **No automated backup or disaster-recovery tooling.** Backups are manual (`pg_dump`).
12. **Demo setup not run on a clean machine.** The full evaluator demo was run end-to-end from the packaged source (48 of 48 checks passed), but only on the development machine, where Docker and base images were already present.
13. **Security review dates.** The 18-phase security review was completed on 6–8 Sep 2026. Features added later (the reworked 5-minute response window, Suggested Solutions, the evaluator demo) are covered by the current unit tests but have not had a dedicated security review.

33. Roadmap

All `PLANNED`: a self-hosted model on suitable hardware; isolation on physical Windows and Linux devices plus a "release isolation" action; a GA Wazuh release and an end-to-end Manager integration; production deployment (domain, CA TLS, secret manager, automated backups, DR, monitoring); dependency major upgrades (NestJS, Vite, Electron); wider detection coverage; full Arabic translation; employee-level notifications; load testing; a dedicated security review of post-8-September features.

34. Glossary

Term	Meaning
SOC	Security Operations Center: the people, process and tools that detect and respond to security incidents.
SMB	Small and medium-sized business.
SIEM / XDR / EDR	Security information and event management / extended detection and response / endpoint detection and response.
Wazuh	An open-source security platform (agents, manager, indexer, dashboard).
Active Response	Wazuh's mechanism for running a command on an agent in response to an event.
Suricata / Zeek	Open-source network intrusion-detection and network-analysis tools.
Sysmon	A Windows system-monitoring driver that logs process, network and file activity.
MITRE ATT&CK	A public knowledge base of adversary tactics and techniques (for example T1566 Phishing).
IOC / IOA	Indicator of compromise (static artifact such as an IP or hash) / indicator of attack (behaviour pattern).
C2	Command and control: attacker communication with compromised hosts.
Persistence	Techniques attackers use to survive reboots (scheduled tasks, services, run keys…).
Correlation	Grouping related events into one incident (case).
Case ID	MIRQAB's incident identifier, `INC-YYYYMMDD-NNNN`.
Response window	The 5-minute period in which a HIGH/CRITICAL incident should be acknowledged.
Containment / isolation	Cutting a compromised endpoint's network access to stop spread.
RBAC	Role-based access control.
IDOR / BOLA	Insecure direct object reference / broken object-level authorization.
JWT	JSON Web Token, used for API authentication.
Idempotent	Safe to repeat: doing it twice has the same effect as once.
Advisory lock	A PostgreSQL lock used to make one decision exactly once across processes.
Digital Twin	MIRQAB's live visual model of the organization's network, devices and active attacks.
Self-hosted AI	A language model run inside the organization's own network instead of an external cloud service.
Prisma	The TypeScript ORM and migration tool used for PostgreSQL.
RFC 5737	IP address ranges reserved for documentation (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24).